HTTP IN DETAIL

TryHackMe Learning & Practical Report

Prepared by: Kaustubh Ashish Bangar

Focus: Web Application Security / WAPT Foundations

Platform: TryHackMe — Cyber Security 101

Room: Web Application Basics

A structured technical report documenting the concepts studied and practical exercises completed in the HTTP in Detail room.

Prepared for learning documentation, portfolio use, and cybersecurity training for the IqraSec Academy WAPT Internship Phase 1.

1. Introduction

The Hypertext Transfer Protocol (HTTP) is the fundamental protocol that powers the World Wide Web. It dictates how messages are formatted and transmitted, and what actions web servers and browsers should take in response to various commands. While the *Web Application Basics* module provided a high-level overview of client-server architecture, a deep, technical understanding of HTTP is strictly required for Web Application Penetration Testing (WAPT). Security analysts must be able to manually read, interpret, and manipulate raw HTTP traffic to uncover vulnerabilities that automated scanners might miss.

This report documents a deep dive into the HTTP protocol, covering its underlying mechanics, request and response structures, headers, state management, and the crucial cryptographic upgrades provided by HTTPS and TLS.

2. Objective

The primary objective of this module is to transition from a conceptual understanding of web applications to a protocol-level mastery of HTTP communication. Specific objectives include:

1. To understand the foundational mechanics of HTTP and its client-server execution model.
2. To dissect the exact syntax and structure of HTTP requests and responses.
3. To deeply analyze HTTP methods and their specific use cases in application logic.
4. To categorize and interpret HTTP status codes (1xx - 5xx) to accurately gauge server behavior.
5. To analyze standard and custom HTTP headers for information disclosure and security policy enforcement.
6. To understand the cryptographic handshake and security benefits of HTTPS and TLS.
7. To study state management mechanisms, specifically cookies and sessions, and their associated security risks.
8. To apply protocol theory through practical exercises and traffic analysis.

3. HTTP Fundamentals

HTTP is an application-layer protocol (Layer 7 in the OSI model) designed to transfer information between networked devices. It operates on a client-server model where the client (usually a web browser) initiates a connection and sends a request, and the server (such as Apache, Nginx, or IIS) listens on a specific port (default is 80 for HTTP and 443 for HTTPS) to process the request and return a response.

A key characteristic of HTTP is that it is a *stateless* protocol. This means the server does not inherently retain any memory of past requests; every request is treated as an entirely new and independent transaction. While this stateless design makes HTTP highly scalable and efficient, it requires developers to implement external mechanisms (like cookies) to track users across multiple requests (e.g., keeping a user logged in as they navigate between pages).

4. HTTP Request Structure

An HTTP request is a highly structured text message sent by the client. It consists of three primary components:

- **Request Line:** The first line of the message. It contains three space-separated elements:
  - The **HTTP Method** (e.g., `GET`, `POST`, `OPTIONS`).
  - The **Request Target** or Path (e.g., `/index.html` or `/api/login`).
  - The **HTTP Version** (e.g., `HTTP/1.1` or `HTTP/2`).
  - *Example:* `GET /dashboard HTTP/1.1`
- **Headers:** Key-value pairs that immediately follow the request line. They provide the server with vital metadata about the client, the expected response format, and the host being requested.
- **Message Body:** Separated from the headers by a single blank line (CRLF). The body is optional and is typically used with `POST` or `PUT` requests to send data to the server, such as form submissions, file uploads, or JSON payloads.

5. HTTP Response Structure

When the server finishes processing a request, it replies with an HTTP response, which perfectly mirrors the request structure:

- **Status Line:** The first line of the response, containing:
  - The **HTTP Version** (e.g., `HTTP/1.1`).
  - The **Status Code** (a three-digit integer, e.g., `200`).
  - The **Reason Phrase** (a short, human-readable explanation, e.g., `OK`).
  - *Example:* `HTTP/1.1 404 Not Found`
- **Headers:** Similar to request headers, response headers provide metadata about the server, security policies, and the type of content being returned.
- **Message Body:** Separated from the headers by a blank line, this contains the actual requested resource (e.g., the HTML source code of a webpage, an image file, or an API error message).

6. HTTP Methods

HTTP methods (also known as verbs) indicate the desired action the client wishes to perform on the target resource.

- **GET:** Requests a representation of the specified resource. GET requests should only retrieve data and must not contain a message body or cause a state change on the server.
- **POST:** Submits an entity to the specified resource, often causing a change in state or side effects on the server (e.g., creating a new user or submitting a payment).
- **PUT:** Replaces all current representations of the target resource with the uploaded content.
- **DELETE:** Deletes the specified resource.
- **OPTIONS:** Returns the HTTP methods that the server supports for the specified URL, highly useful during the reconnaissance phase of a penetration test.
- **HEAD:** Asks for a response identical to a GET request, but without the response body. Useful for checking if a file exists or its size without downloading the entire file.

7. HTTP Status Codes

Status codes are three-digit integers grouped into five distinct classes, providing immediate diagnostic feedback during WAPT:

- **1xx (Informational):** The request was received, and the process is continuing (e.g., `101 Switching Protocols`).
- **2xx (Successful):** The request was successfully received, understood, and accepted (e.g., `200 OK`, `201 Created`).
- **3xx (Redirection):** Further action must be taken to complete the request (e.g., `301 Moved Permanently`, `302 Found`).
- **4xx (Client Error):** The request contains bad syntax or cannot be fulfilled due to client-side issues. Critical for testers (e.g., `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`).
- **5xx (Server Error):** The server failed to fulfill a valid request due to a backend issue (e.g., `500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`). In WAPT, 500 errors often indicate poorly handled input, hinting at potential vulnerabilities like SQL injection.

8. HTTP Headers

Headers control the nuances of the HTTP transaction.

- **Host:** Mandatory in HTTP/1.1. It specifies the domain name of the server. Crucial for servers hosting multiple websites on a single IP address (Virtual Hosting).
- **User-Agent:** Identifies the client software, OS, and device type. Attackers often manipulate this to bypass basic access controls or trigger mobile-specific application logic.
- **Accept:** Tells the server what media types the client can process (e.g., `text/html`, `application/json`).
- **Content-Length:** Indicates the size of the message body in bytes. Manipulating this header is a core technique in HTTP Request Smuggling attacks.
- **Authorization:** Contains credentials to authenticate a user with a server, often carrying Bearer tokens or Basic Auth strings.

9. HTTPS and TLS Fundamentals

Because standard HTTP is transmitted in plaintext, it is vulnerable to packet sniffing and Man-in-the-Middle (MitM) attacks. HTTPS (HTTP over TLS/SSL) solves this by encrypting the communication channel.

- **TLS (Transport Layer Security):** A cryptographic protocol that provides end-to-end security over networks. It ensures three core principles:
    1. **Encryption:** Hides the data from third parties.
    2. **Authentication:** Ensures the parties exchanging information are who they claim to be via Digital Certificates.
    3. **Integrity:** Verifies that the data has not been forged or tampered with during transit. During the TLS Handshake, the client and server agree on cryptographic algorithms and exchange keys before any HTTP data is transmitted, securing all headers, paths, and bodies.

10. Cookies and Sessions

To overcome HTTP's stateless nature, servers utilize sessions and cookies.

- **Sessions:** Server-side storage of user data. When a user logs in, the server creates a unique session ID.
- **Cookies:** The server sends this session ID to the browser via the `Set-Cookie` header. The browser stores it and automatically appends it to the `Cookie` header in all future requests to that domain.
- **Security Vulnerabilities:** If an attacker steals a user's session cookie (via XSS or network interception), they can completely hijack the user's account. Therefore, securing cookies with the `HttpOnly`, `Secure`, and `SameSite` flags is a critical security requirement.

11. Practical Exercises Completed

The practical application for this module required intercepting, analyzing, and manually interacting with raw HTTP traffic to solidify the theoretical concepts.

- **Traffic Analysis:** Analyzed raw HTTP requests and responses to identify specific headers, methods, and status codes.
- **Method Manipulation:** Issued requests using non-standard methods (like OPTIONS) to enumerate allowed server actions.
- **Header Modification:** Modified `User-Agent` and `Host` headers to observe changes in server behavior and bypass simulated client restrictions.
- **Status Code Interpretation:** Triggered various client and server errors (`404 Not Found`, `403 Forbidden`, `500 Internal Server Error`) by requesting restricted directories or supplying malformed data, interpreting the results to map the application's attack surface.

12. Tools Used

- TryHackMe Platform Environment
- Web Browser Developer Tools (Network Tab)
- In-Browser API manipulation tool

13. Evidence and Screenshots

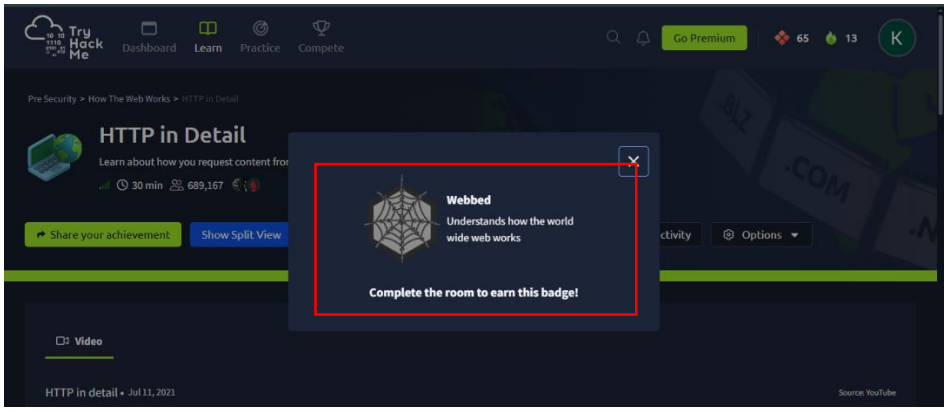


Fig. 2.1: TryHackMe HTTP in Detail room completion certificate

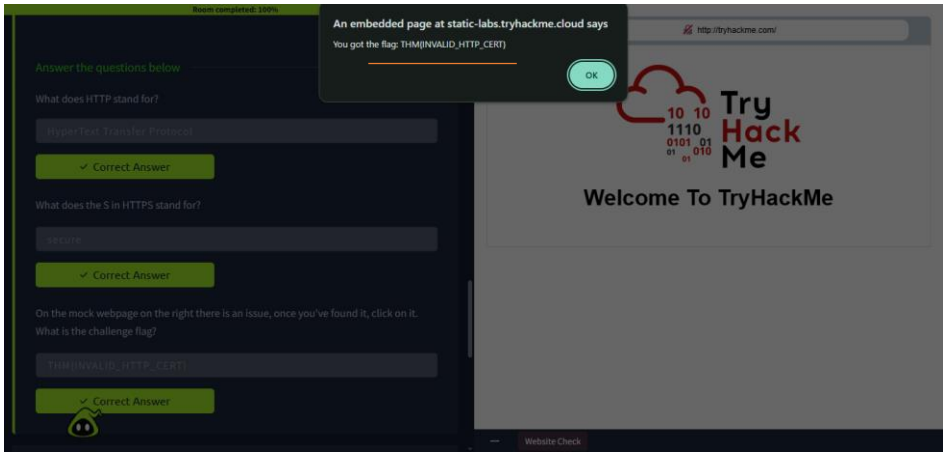


Fig. 2.2: Identifying an invalid HTTP certificate issue on the mock webpage

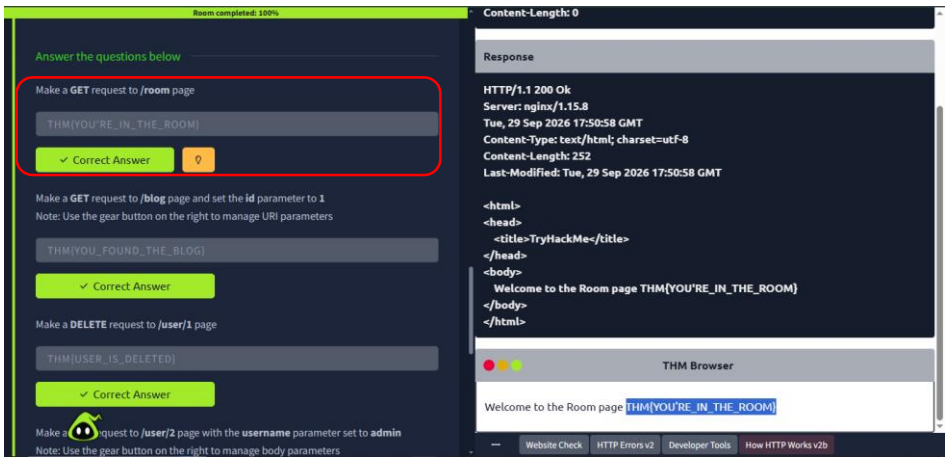


Fig. 2.3: Successful execution of a GET request to the `/room` page to retrieve data

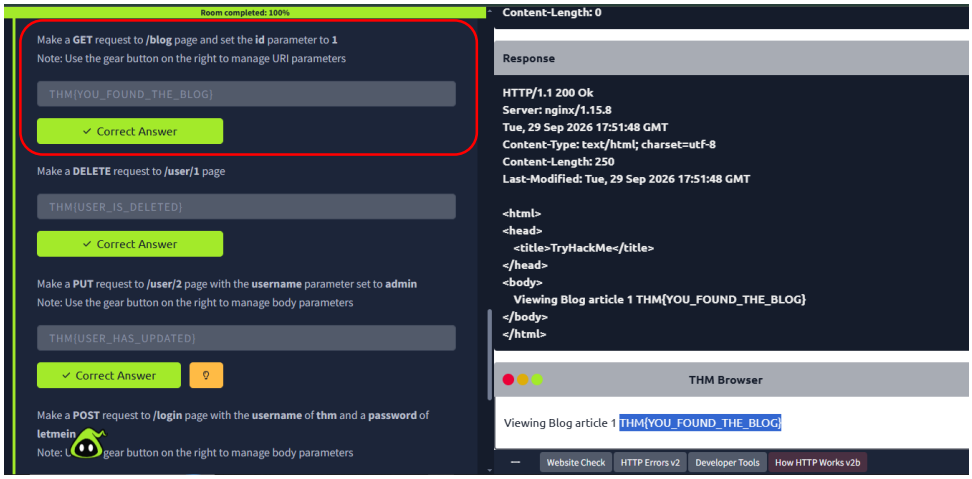


Fig. 2.4: Execution of a GET request to the `/blog` page utilizing the `id=1` URI parameter

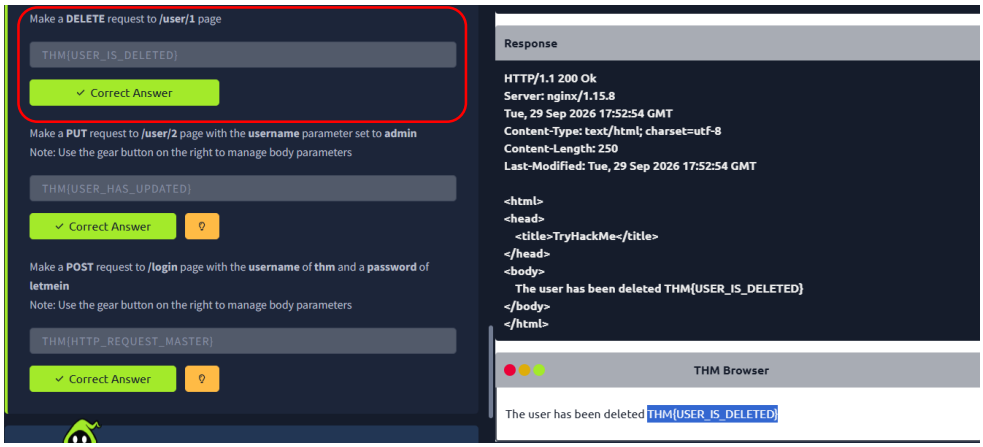


Fig. 2.5: Successful DELETE request executed against the `/user/1` page to remove a resource

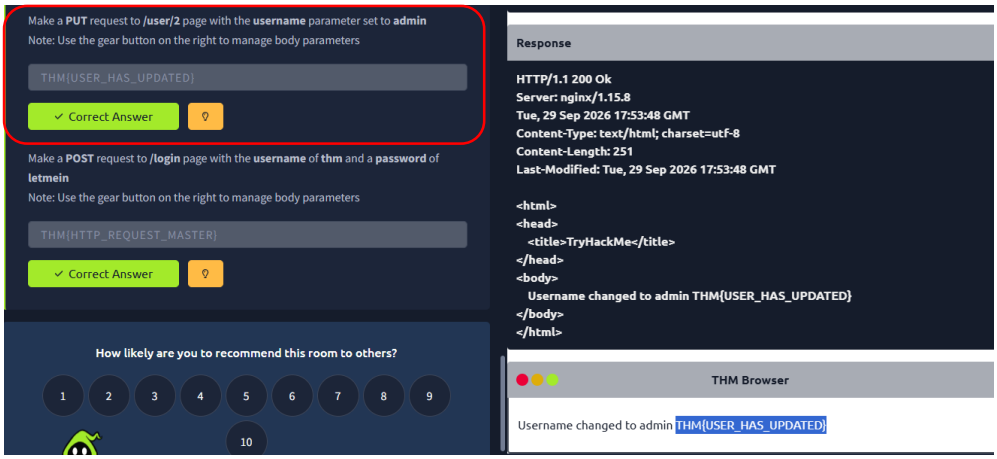


Fig. 2.6: PUT request targeting `/user/2` with the body parameter `username=admin` to update the resource

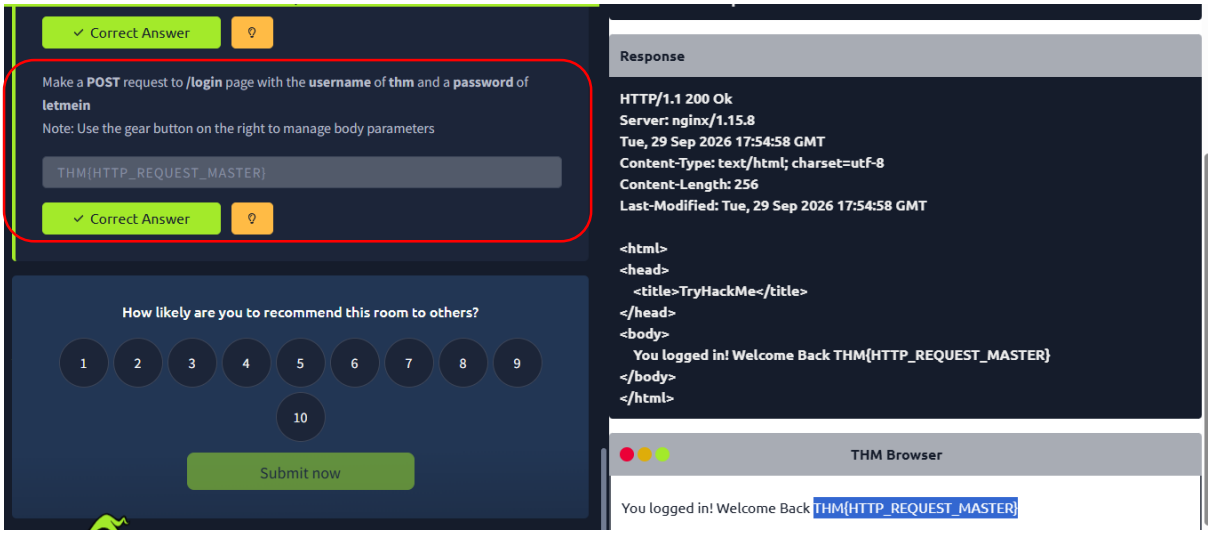


Fig. 2.7: POST request submitted to the `/login` endpoint with `username` and `password` body parameters to simulate authentication

14. Results

The HTTP in Detail module was completed successfully. I gained a highly technical, protocol-level understanding of how data is structured and transmitted across the web. The practical exercises successfully demonstrated how manipulating raw HTTP requests—bypassing the browser entirely—is a fundamental technique for identifying logical flaws and security misconfigurations in web applications.

15. Key Learnings

- **Statelessness is a Security Challenge:** Because HTTP is stateless, the mechanisms used to track state (cookies, tokens) are prime targets for attackers. Flaws in session management often lead to complete account compromise.
- **Headers Dictate Behavior:** Headers are not just metadata; they control security policies (like CORS and CSP) and tell the server how to parse payloads. Tampering with headers like `Content-Type` or `Content-Length` can exploit parser vulnerabilities.
- **Status Codes are Intelligence:** An unexpected `500 Internal Server Error` is often a strong indicator that the application does not properly handle malicious input, making it a critical starting point for SQL Injection or Command Injection testing.
- **HTTPS is Mandatory, but not a Silver Bullet:** While HTTPS protects data in transit from network-level sniffing, it does *not* protect the application from logical

vulnerabilities (like XSS or IDOR) because the malicious payload is decrypted by the server before processing.

16. Conclusion

Completing the HTTP in Detail room provided the precise technical depth required for professional Web Application Penetration Testing. Understanding the exact syntax of requests and responses, the purpose of specific headers, and the mechanics of state management bridges the gap between simply using automated security tools and actually understanding *why* a vulnerability exists. This deep protocol knowledge is the direct foundation required for mastering traffic interception tools like Burp Suite and effectively testing for complex, multi-stage vulnerabilities in Phase 1 and beyond.

17. References

1. TryHackMe - HTTP in Detail: https://tryhackme.com/room/httpindetail
2. WAPT Phase 1 Task Instructions - IqraSec Academy